OS Orange Problem-2

Name: PRANAV SP

SRN: PES1UG24CS337

SECTION: 4F

Phase 1: Object Storage Foundation

1A

```
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b
9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$
```

1B

```
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$
```

Phase 2: Tree Objects

2A

```
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$
```

2B

```
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ xxd .pes/objects/d5/8213f
5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12 | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$
```

Phase 3: The Index (Staging Area)

3A

```
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ ./pes init
Initialized empty PES repository in .pes/
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ echo "hello" > file1.txt
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ ./pes add file1.txt
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 17765
16039 6 file1.txt
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$
```

3B

```
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ ./pes init
Initialized empty PES repository in .pes/
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ echo "hello" > file1.txt
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ ./pes add file1.txt
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 17765
16039 6 file1.txt
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$
```

Phase 4: Commits and History

4A

```
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ ./pes log
commit 86bd6a46bdff48602e83b9465d709db976fd2d3cc0c3eaf26cb3830172bf5606
Author: Pranav SP <PES1UG24CS337>
Date:   1776520374

    third commit

commit 76a0f0f276f88489aad59f297b6109fe2fa7350de74bcb1f4eaee95853d72b45
Author: Pranav SP <PES1UG24CS337>
Date:   1776520336

    second commit

commit 9579b67014cd123c43e149b19736de24831ac864b5c75dec2c1a8b683f94153f
Author: Pranav SP <PES1UG24CS337>
Date:   1776520262

    first commitirst commiQ

pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$
```

4B

```
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/16/84c63c50d13046275f0188f9d0cdb4a1c82917b2f44b23d07d072b1ab9fc0
b
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb
4
.pes/objects/72/5fcc4e398378423b92e9d31fb67808b3a99c59126336c89a9d6d792f082cf
f
.pes/objects/76/a0f0f276f88489aad59f297b6109fe2fa7350de74bcb1f4eaee95853d72b4
5
.pes/objects/86/bd6a46bdff48602e83b9465d709db976fd2d3cc0c3eaf26cb3830172bf560
6
.pes/objects/95/79b67014cd123c43e149b19736de24831ac864b5c75dec2c1a8b683f94153
f
.pes/objects/f2/bff1edf58f9ab3c2da22921747d504b737f5acc1cd780f541b0e06775505d
e
.pes/refs/heads/main
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$
```

4C

```
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ cat .pes/refs/heads/main
86bd6a46bdff48602e83b9465d709db976fd2d3cc0c3eaf26cb3830172bf5606
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ cat .pes/HEAD
ref: refs/heads/main
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$
```

FINAL

```
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
```

```
Untracked files:
  (nothing to show)


--- First Commit ---
Committed: cd038b4876b4... Initial commit

Log after first commit:
commit cd038b4876b47583ad68343f41098c0705615d28b5944b8976a84506c375a803
Author: Pranav SP <PES1UG24CS337>
Date:   1776520660

    Initial commit


--- Second Commit ---
Committed: 7fb0517a30c9... Update file.txt

--- Third Commit ---
Committed: 5040f0bb5820... Add farewell
```

```
--- Full History ---
commit 5040f0bb5820dc2def733b3966b329fcad464dec49274808d21eafa1bcbac3b2
Author: Pranav SP <PES1UG24CS337>
Date:   1776520660

    Add farewell

commit 7fb0517a30c9a0511efcd1da9f2c5654e1e7dd148a92b90a0d47ca328b17c2b3
Author: Pranav SP <PES1UG24CS337>
Date:   1776520660

    Update file.txt

commit cd038b4876b47583ad68343f41098c0705615d28b5944b8976a84506c375a803
Author: Pranav SP <PES1UG24CS337>
Date:   1776520660

    Initial committial comQ
```

```
--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
5040f0bb5820dc2def733b3966b329fcad464dec49274808d21eafa1bcbac3b2

--- Object Store ---
Objects created:
8
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/16/84c63c50d13046275f0188f9d0cdb4a1c82917b2f44b23d07d072b1ab9fc0b
.pes/objects/50/40f0bb5820dc2def733b3966b329fcad464dec49274808d21eafa1bcbac3b2
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/7f/b0517a30c9a0511efcd1da9f2c5654e1e7dd148a92b90a0d47ca328b17c2b3
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/cd/038b4876b47583ad68343f41098c0705615d28b5944b8976a84506c375a803
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
pranav@pranav-VirtualBox:~/Desktop/OS2/OS-orange-2$
```

Phase 5 & 6: Analysis

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

Ans: Update `.pes/HEAD` to point to `refs/heads/<branch>` and read the commit hash from that branch file. Then load the commit's tree and update the working directory to match it (create, modify, or delete files as needed), and update the index accordingly.

This is complex because we must handle overwriting files, deleting old ones, and avoid losing any uncommitted changes in the working directory.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Ans: We check each file in the index and compare it with the working directory using `stat()`. If the size or timestamp is different, it means the file was modified. Then we check the same file in the target branch (from the object store). If that version is also different, it means checkout will overwrite it. So in that case, we stop the checkout to avoid losing the user's changes.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Ans: In detached HEAD, commits still get created normally, but no branch points to them. So they're kind of "floating" and can get lost if you switch branches later.

To recover them, you can use the commit hash (from log) or just create a new branch pointing to that commit.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Ans: We start from all branch heads and traverse every commit, tree, and blob, marking each visited object as "reachable". After traversal, we delete any object in `.pes/objects` that was not marked.

We can use a hash set to store visited hashes for fast lookup.

Even with 50 branches, most history is shared, so we'd visit roughly 100k–300k objects, not 100k × 50.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Ans: Running GC during a commit is risky because the commit may create objects (like blobs/trees) that are not yet linked from any branch. GC might see them as unreachable and delete them.

For example, a commit writes a tree object but hasn't updated HEAD yet. At the same time, GC runs and deletes that tree since no commit points to it yet. Then the commit finishes and references a missing object → corruption.

Git avoids this using locking, writing objects atomically, and not deleting recently created objects immediately.

Github repo link- https://github.com/pranavsp12-dev/OS-orange-2